

VS Code Alıştırma Rehberi — 2 Günde "Kodun Sahibi" Ol

Amaç: Çarşamba 8:30'da hoca "şunu VS Code'da göster / şunu değiştir / bunu nasıl yaptın?" dediğinde tereddütsüz yapabilmek. Bütün alışırmalar KENDİ projen üzerinde. Kod zaten baştan sona Türkçe yorumlu — okuma yükün düşük; iş, elinin alışması.

Önce güvenlik ağını öğren (5 dk): Kodu bozmaktan KORKMA, her şey geri alınabilir:

- **Ctrl+Z** = son değişikliği geri al (dosya içinde).
- Dosyayı tamamen eski haline döndürmek: terminalde `git checkout -- dosya/yolu.py`
- HER ŞEYİ eski haline döndürmek: `git checkout -- .`
- Kendi kuralın: hocanın önünde değişiklik yaptıysan, gösterim bitince **Ctrl+Z** ile geri al ve kaydet.

GÜN 1 — SABAH: VS Code'da Gezinmeyi Refleks Yap (1,5 saat)

Bu 6 kısayolu bu projede 3'er kez uygula — sunumda lazım olacak olanlar sadece bunlar:

Kısayol	Ne yapar	Bu projede alıştırma
Ctrl+P	Dosya adıyla aç	Yaz: <code>training</code> → Enter → <code>training.py</code> açılır. Tekrarla: <code>models</code> , <code>demo</code>
Ctrl+Shift+F	TÜM projede metin ara	Ara: <code>patience</code> → hangi dosyalarda geçtiğini gör. Tekrarla: <code>n_intervals</code> , <code>early</code>
Ctrl+F	Açık dosyada ara	<code>training.py</code> içinde ara: <code>inverse_frequency</code>
Ctrl+G	Satıra git	Bir dosyada 100. satıra atla
F12	Tanıma git (fonksiyonun kaynağına)	<code>pipeline.py</code> 'de <code>train_with_early_stopping</code> çağrısının üstünde F12 → tanımına ışlanırsın
Ctrl+ö (TR klavye)	Terminali aç/kapa	Aç, <code>dir final</code> yaz, kapat, tekrar aç

Hoca karşısındaki en havalı hareket: "X nerede?" sorusuna Ctrl+Shift+F ile 2 saniyede inmek. Bunu refleks yapana kadar tekrarla.

GÜN 1 — ÖĞLEN: Rehberli Kod Turu (2,5 saat)

Sıra önemli — veri akışının sırası. Her dosyada: üstteki açıklamayı oku → görevleri yap.

1) final/features.py

40 dk

1. Baştan sona oku (yorumlar Türkçe).
2. Bul (Ctrl+F): `n_intervals` ve `interval_ms` — ödevin iki hiperparametresinin TANIMLANDIĞI yer burası. Hocaya burayı göstereceksin.
3. Bul: `feature_dim` — 44 sayısının formülü ($3 \times 13 + 5$). Sesli açıkla: "13 MFCC ortalama + 13 std + 13 delta + 5 istatistik".
4. Bul: `fix_frames` — kırpma/dolgu mantığını kendi cümlele anlat.

2) final/models.py

40 dk

1. Bul: `class ConvBlock` — içindeki 7 katmanı sırayla say (Conv-BN-ReLU-Conv-BN-ReLU-MaxPool).
2. Bul: `channels: tuple` — "bir katman daha ekle" isteği buraya (32,64,128) → (32,64,128,256) yazmak demek. ŞİMDİ DENE, sonra Ctrl+Z.
3. Bul: `class SeqRNN` — `bidirectional` ve `pooling` satırlarını bul; forward'daki 4 havuzlama dalını gör.
4. Bul: `patience: int = 8` (OptimSettings içinde) — erken durdurma sabrının evi.

3) final/training.py

45 dk — EN ÖNEMLİ DOSYA

1. Bul: `inverse_frequency_weights` — ağırlık formülü satırı: `weights = len(labels) / (num_classes * counts...)`. Hoca "ağırlıklı loss nerede?" derse ADRES BURASI.
2. Bul: `epochs_without_improvement >= optim.patience` — ERKEN DURDURMANIN kendisi bu satır. Bir üstündeki `best_state` saklama bloğunu da göster.
3. Bul: `model.load_state_dict(best_state)` — "en iyi epoch'a geri dönüş" satırı.
4. Bul: `train_transform` — SpecAugment'in SADECE eğitim yığınınına uygulandığı yer.

4) final/pipeline.py + search_space.py

35 dk

1. `search_space.py`'de bul: `n_intervals=16` — aralık adaylarının listelendiği blok. "Aramaya nasıl kattın?" sorusunun cevabı.
2. `pipeline.py`'de bul: `test fold is touched exactly once` yorumunun oralar — test disiplini kodu.
3. `pipeline.py`'de bul: `Standardizer.fit(train_x` — normalizasyonun YALNIZ train'den öğrenildiği satır.

GÜN 1 — AKŞAM: Çalıştırma Pratiği (1 saat)

Terminalden sırayla çalıştır (Ctrl+ö):

```
.env\Scripts\python.exe -m pytest final\tests -q
.env\Scripts\python.exe -X utf8 final\demo.py --rastgele 3
.env\Scripts\python.exe -X utf8 final\demo.py final\demo_ornekleri\1013_TIE_DIS_XX.wav
```

İlki 18 testin geçtiğini gösterir ("kodum test edilmiş" kanıtı). Diğer ikisi demo. Üçünü de EZBERE yazabilene kadar tekrarla — sunumda titremeden yazacaksın.

Notebook'u aç: Ctrl+P → SONUCLAR → kernel olarak .venv seç → Run All → tekrar-üretim tablosundaki "birebir aynı mı: True" satırını bul. Hoca "tekrar üretilebilir mi?" derse CANLI KANIT bu.

GÜN 2 — HOCA SİMÜLASYONU: 10 Drill

Her drill = muhtemel bir hoca isteği. Formatı: İSTEK → NE YAPACAKSIN → NE SÖYLEYECEKSİN. Hepsini en az bir kez ELİNLE yap. Değişiklik gerektirenlerde sihirli cümle şu: **"Gerçek eğitim ~40 dakika sürüyor hocam; mantığı 2 dakikalık hızlı modda göstereyim"** — quick mod tam bunun için var.

DRILL 1 — "Aralık sayısını 16 yap, ne oluyor görelim"

1. Ctrl+P → features.py → Ctrl+F `n_intervals` → 24 'ü 16 yap → Ctrl+S.
2. Terminal: `.venv\Scripts\python.exe final\run_experiment.py --grid-mode quick --limit-per-split 60 --methods rnn`
3. ~2 dk sonra val skorları gelir. SÖYLE: "Varsayılan aralık sayısını değiştirdim; hızlı modda etkisini görüyoruz. Tam aramada 16'lık aday zaten denendi ve 32'ye yenildi."
4. Bitince: Ctrl+Z → Ctrl+S (geri al!).

DRILL 2 — "Patience'ı 3'e düşür"

1. models.py → Ctrl+F `patience: int = 8` → 3 yap → kaydet.
2. Aynı quick komutu koş; loglarda "epoch x/y" değerlerinin küçüldüğünü göster.
3. SÖYLE: "Sabır düşünce eğitim daha erken kesiliyor; geçici dalgalanmada erken pes etme riski artar — 8'i o dengeden seçtim." Geri al.

DRILL 3 — "CNN'e bir katman daha ekle"

1. models.py → `channels: tuple[int, ...] = (32, 64, 128)` → `(32, 64, 128, 256)` → kaydet.
2. `.venv\Scripts\python.exe final\run_experiment.py --grid-mode quick --limit-per-split 60 --methods cnn`
3. SÖYLE: "Tuple'a eleman eklemek yeni ConvBlock demek — mimari config'den türüyor. Tam aramada 4 bloklu aday denendi ve kazanamadı: veri 5 bin kayıtken büyük ağ ezberliyor." Geri al.

DRILL 4 — "Ağırlıkları gerçekten hesapladığını göster"

```
.venv\Scripts\python.exe -c "print(7442/(6*1087), 7442/(6*1271))"
```

Çıktı: 1.141 ve 0.976. SÖYLE: "Formül training.py'deki `inverse_frequency_weights`; nötr azınlık olduğu için ağırlığı 1'in üstünde." (Fonksiyonu Ctrl+Shift+F ile aç, satırı göster.)

DRILL 5 — "Erken durdurma kodunu göster"

Ctrl+Shift+F → `epochs_without_improvement` → `training.py`'de üç noktayı sırayla göster: sayacın artışı → `>= optim.patience: break` → `load_state_dict(best_state)` . SÖYLE: "Say, dur, en iyiye dön — üç satırlık özeti bu."

DRILL 6 — "Şu ses dosyasını sınıflandır" (hoca kendi dosyasını verebilir)

```
.venv\Scripts\python.exe -X utf8 final\demo.py "DOSYA\YOLU.wav"
```

Herhangi bir .wav yolu çalışır (16 kHz'e otomatik çevrilir). Bir kez kendi Müzik klasöründen rastgele bir wav ile dene ki formata güvenin olsun.

DRILL 7 — "Mel bant sayısını 32 yap"

`features.py` → `n_mels: int = 64` → 32 → quick CNN koşusu → SÖYLE: "Çözünürlük düşünce bilgi kaybı var; 80'lik ince varyantı da aramada denedim, 64'ü geçemedi." Geri al.

DRILL 8 — "Testleri çalıştır"

```
.venv\Scripts\python.exe -m pytest final\tests -q
```

18 yeşil. SÖYLE: "Öznitelik boyutları, önbellek, modellerin ileri geçişi ve normalizasyon eksenleri test altında."

DRILL 9 — "GRU yerine LSTM olsa?"

`models.py` → `rnn_type: str = 'gru'` → `'lstm'` → quick RNN koşusu. SÖYLE: "Tek kelimelik değişiklik — tasarım gereği. Tam aramada LSTM adayı vardı, GRU önde bitirdi: benzer güç, daha az parametre." Geri al.

DRILL 10 — "Sonuç grafiğini/karışıklık matrisini aç"

Ctrl+P → `confusion` → `test_confusion_matrix.png` VS Code içinde açılır. Aynı şekilde `winner_learning_curve.png` , `search_log.csv` (VS Code CSV'yi tablo gibi gösterir). "Bütün çıktılarımla final/outputs altında, dosya olarak duruyor."

KOPYA KÂĞIDI — "X nerede?" Tablosu (yazdır, yanında dursun)

Hoca sorarsa	Dosya	Aranacak kelime (Ctrl+Shift+F)
Aralık hiperparametreleri	features.py / search_space.py	n_intervals
Ağırlıklı loss	training.py	inverse_frequency
Erken durdurma	training.py	epochs_without_improvement
En iyi epoch'a dönüş	training.py	load_state_dict(best_state)
Normalizasyon (sızıntı yok)	dataset.py / pipeline.py	Standardizer.fit
Konuşmacı-bağımsız bölme	ser/data/splits.py	_speaker_partition
CNN mimarisi	models.py	class MelCNN
GRU + havuzlama	models.py	class SeqRNN
SpecAugment	augment.py	class SpecMask
Test disiplini	pipeline.py	touched exactly once
Metrikler	ser/evaluate.py	compute_metrics
Demo	demo.py	run_demo

2 günün planı:

Bugün: Gün 1'in tamamı (gezinme + kod turu + çalıştırma) + akşam SUNUM_KONUSMA_METNİ'nden 1 sesli prova.

Yarın: Sabah 10 drill'i elinle yap → öğlen Claude'la hoca simülasyonu (aşağıya bak) → akşam tam prova (sunum + demo, süre tutarak) → laptopta her şeyin kurulu olduğunu son kez doğrula.

Claude'la simülasyon: Bilgisayarında bu sohbete "hoca simülasyonu başlat" yaz — ben hoca gibi rastgele istekler sorayım ("aralığı değiştir", "şu fonksiyonu göster"), sen VS Code'da yap, "yaptım" de, ben dosyadan kontrol edip geri bildirim vereyim. Gerçek prova bundan iyisi olmaz.

Sunum günü altın kural: Hoca değişiklik isterse acele etme: "tabii hocam" de → Ctrl+Shift+F ile bul → değiştir → quick komutunu koş → sonucu yorumla → **Ctrl+Z ile geri al**. Bilmediğin çıkarsa: "Şurada olması lazım, bakıyorum" deyip arama yap — aramayı bilmen, ezbere bilmenden daha güven verici görünür.